

# Gym-PyBullet: run the drones

**Terminal:** Windows PowerShell

This guide uses the existing project environments and installed simulators. The learned controller uses simulator-trained dynamics with dual-quaternion pose error. Run one simulation at a time.

## 1. Open the project folder

```
cd "D:\drones\Dual-Quaternion-Based-Control-for-a-Leader-Follower-Formation-of-Two-Quadrotors"
```

## 2. Train only if needed

The three backend models were generated in the previous update. Skip training when the matching model exists. Retrain after changing backend dynamics; training runs without a GUI.

```
.\.venv\Scripts\python.exe -B scripts/train_data_driven.py `
  --simulator pybullet
```

## 3. Launch the figure-eight flight

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator pybullet --controller learned `
  --trajectory lemniscate --follow_distance 0.8 `
  --duration 60 --gui
```

The follower targets the leader's recorded path 0.8 m behind. Red trails show the leader; blue trails show the follower. Tracking has numerical error; the drones do not occupy the same position simultaneously.

## 4. Stop or finish

Wait for the configured duration, or press **Ctrl+C** in the terminal. An interrupted run may not save a result. Let the previous command finish before starting another simulation.

**Copying commands:** PowerShell lines use a backtick ( ` ) at the end for continuation. Paste the complete block; do not add spaces after the backtick. No environment activation is needed because the commands use its Python executable directly.

# MuJoCo: run the drones

**Terminal:** Windows PowerShell

## 1. Open the project folder

```
cd "D:\drones\Dual-Quaternion-Based-Control-for-a-Leader-Follower-Formation-of-Two-Quadrotors"
```

## 2. Train if the MuJoCo model is missing

```
.\.venv\Scripts\python.exe -B scripts/train_data_driven.py `
--simulator mujoco
```

## 3. Launch the figure-eight flight

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
--simulator mujoco --controller learned `
--trajectory lemniscate --follow_distance 0.8 `
--duration 60 --gui
```

## 4. Run a B-spline trajectory instead

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
--simulator mujoco --controller learned `
--trajectory bspline --follow_distance 0.8 `
--duration 60 --gui
```

**Current limitation:** the MuJoCo backend stabilizes attitude but does not follow requested yaw. Its successful position tracking must not be described as full pose-tracking validation.

## Model files are simulator-specific

MuJoCo uses **models/mujoco\_response.npz**. PyBullet uses **models/pybullet\_response.npz**. Do not copy or rename a model from another backend to bypass a missing-model error.

For a run without a window, remove **--gui**. To hide the red/blue path lines, add **--no-trails**. Press **Ctrl+C** to stop early.

# Gazebo: run through WSL

**Terminal:** PowerShell first, then Ubuntu / WSL

## 1. Enter Ubuntu from Windows PowerShell

```
wsl -d Ubuntu-24.04
```

Run every command below in the Ubuntu shell that opens, not in PowerShell.

## 2. Open the project and load ROS + Python

```
cd /mnt/d/drones/Dual-Quaternion-Based-Control-for-a-Leader-Follower-Formation-of-Two-Quadrotors
source /opt/ros/jazzy/setup.bash
source .venv-gazebo/bin/activate
```

## 3. Train if the Gazebo model is missing

```
python -B scripts/train_data_driven.py --simulator gazebo
```

## 4. Launch with the WSL graphics settings

```
env -u QT_QUICK_BACKEND QT_QPA_PLATFORM=xcb \
  LIBGL_ALWAYS_SOFTWARE=1 \
  python -B scripts/run_experiment.py \
  --simulator gazebo --controller learned \
  --trajectory lemniscate --follow_distance 0.8 \
  --duration 60 --gui --gazebo_render_engine ogre
```

## 5. If the GUI is black, check a headless run

```
python -B scripts/run_experiment.py \
  --simulator gazebo --controller learned \
  --trajectory lemniscate --follow_distance 0.8 --duration 60
```

Gazebo and ROS 2 Jazzy must already be installed. Headless flight was verified; WSL GUI rendering was not reverified. Runtime log paths are printed in the terminal. A successful headless run does not prove the graphics window is working.

Ubuntu multiline commands use a backslash (\), not a PowerShell backtick. Repeat step 2 in every new Ubuntu terminal. Press **Ctrl+C** to stop the current run.

# Trajectories and control options

**Terminal:** Windows PowerShell examples; start from the project folder

These examples use PyBullet. Replace **pybullet** with **mujoco** to use its trained model. For Gazebo, change the same flags in page 3's Ubuntu command and keep its graphics prefix.

## 3D potato-chip route

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator pybullet --controller learned `
  --trajectory potato_chip --duration 60 --gui
```

## B-spline route

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator pybullet --controller learned `
  --trajectory bspline --duration 60 --gui
```

## LERP / SLERP pose interpolation

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator pybullet --controller learned `
  --trajectory interpolated --duration 30 --gui
```

## Original analytic controller for comparison

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator pybullet --controller analytic --experiment real_eig `
  --trajectory lemniscate --duration 60 --gui
```

Analytic mode needs no trained model and is not the data-driven controller. It still uses recorded-path following unless you explicitly change the formation mode.

## Useful flags to add or replace

- follow\_distance 1.0:** one-metre gap along the recorded route.
- duration 30:** shorter run. **--no-trails:** hide path lines.
- output results/demo:** save this run in a separate folder.
- trajectory\_file configs/course\_bspline.json:** use this B-spline definition.
- kalman --position\_noise\_std 0.03:** optional noisy-position experiment.
- optimize\_path:** optional B-spline smoothing; needs CVXPY and **--trajectory bspline**.

These are runnable options, not a guarantee of accuracy on every route. Keep control at 48 Hz and physics at 240 Hz for the supplied models. Changing these or backend dynamics requires compatible training.

# Training, results and checks

**Terminal:** PowerShell unless a block says Ubuntu

## Refit existing recordings without recollecting flights

```
.\.venv\Scripts\python.exe -B scripts/train_data_driven.py `
  --simulator pybullet --reuse-data
```

Use matching duration and episode settings. Replace pybullet with mujoco for that backend. In Ubuntu, use **python -B** and **--simulator gazebo**. This command reruns evaluation and updates the model/report.

## Check tests and display every available run option

```
.\.venv\Scripts\python.exe -B -m pytest -q
.\.venv\Scripts\python.exe -B scripts/run_experiment.py --help
```

## Plot your latest PyBullet run

```
$run = Get-ChildItem results/pybullet_learned_*.npz |
  Sort-Object LastWriteTime -Descending | Select-Object -First 1
.\.venv\Scripts\python.exe -B scripts/plot_results.py --run $run.FullName
```

First complete a PyBullet run with the default results folder. Replace the filename pattern with **mujoco\_learned\_\*.npz** for MuJoCo.

## Optional ideal simulation and course analysis

```
.\.venv\Scripts\python.exe -B scripts/run_experiment.py `
  --simulator kinematic --controller analytic --duration 30
.\.venv\Scripts\python.exe -B scripts/analyze_course_concepts.py
```

Kinematic mode is an ideal control-law check with no physics-engine GUI. The analysis script provides supporting mathematical demonstrations.

## Where your files are saved

**models/:** simulator-specific trained response models.

**data/identification/<backend>:** training recordings, validation flights and report.json.

**results/:** completed experiment .npz logs.

**results/learned\_validation/summary.json:** saved 60-second validation metrics.

**docs/data\_driven.md:** method, results and limitations.

## If a command fails

**Trained model missing:** run the training command for that exact simulator.

**Cannot find Python or scripts:** repeat the folder command on the simulator's page and check the correct terminal.

**Gazebo ROS import error:** repeat both source commands in Ubuntu.

**Model timestep mismatch:** remove custom frequency flags to use the supplied model settings.

**Module not found:** the required backend dependency is absent from that environment; this guide assumes the existing installation.